Import the datasets

In [94]:
```
print("CLIENT PORTFOLIO shape:", client_portfolio.shape)
```

```
CLIENT PORTFOLIO shape: (1035, 24)
```

In [95]:
```
import pandas as pd
import numpy as np
import os.path as osp
data_path = r"C:\Users\BOC\Desktop\MFIT 860\Project\Data\project_data.xlsx"

# Import the two worksheets
client_portfolio = pd.read_excel(
    data_path,
    sheet_name="CLIENT PORTFOLIO"
)

deposit_account = pd.read_excel(
    data_path,
    sheet_name="DEPOSIT ACCOUNT"
)
print("DEPOSIT ACCOUNT")
display(deposit_account.head())
```

```
DEPOSIT ACCOUNT
```

	CLIENT_NUMBER	DEPOSIT_ACCOUNT_NO	DEMAND_TERM	BAL_XCA	TERM_START_DA
0	10008128852	900603955956	D	5.000000e+05	N
1	10008078231	900600599850	D	9.852871e+06	N
2	10008076417	900600070491	D	4.989743e+04	N
3	10008076504	900600020502	D	3.950350e+04	N
4	10008076596	900600021000	D	8.036230e+03	N

Review the structure of the datasets

In [96]:
```
print("CLIENT PORTFOLIO shape:", client_portfolio.shape)
```

```
CLIENT PORTFOLIO shape: (1035, 17)
```

In [97]:
```
print("DEPOSIT ACCOUNT shape:", deposit_account.shape)
```

```
DEPOSIT ACCOUNT shape: (1142, 11)
```

In [98]:
```
print("\nCLIENT PORTFOLIO columns:")
print(client_portfolio.columns.tolist())
```

```
CLIENT PORTFOLIO columns:
['CLIENT_NUMBER', 'ACCOUNT_OPEN_DATE', 'OFFICE_PROV', 'OFFICE_COUNTRY', 'INDUSTRY',
'KYC_RATING', 'CREDIT_RISK_RATING', 'DEPOSIT_CURRENT_BALANCE', 'DEP_AVE_YEAR_BAL',
'DEP_INT_INCOME', 'DEP_INT_EXP', 'DEP_NET_INT_INC', 'LOAN_CUR_BAL', 'LOAN_INT_INCOM
E', 'LOAN_EXPENSE_INTEREST', 'LOAN_NET_INT_INC', 'FEE_INC']
```

In [99]:
```
print("\nDEPOSIT ACCOUNT columns:")
print(deposit_account.columns.tolist())
```

```
DEPOSIT ACCOUNT columns:
['CLIENT_NUMBER', 'DEPOSIT_ACCOUNT_NO', 'DEMAND_TERM', 'BAL_XCA', 'TERM_START_DATE',
'TERM_END_DATE', 'TERM_DAYS', 'CLIENT_INTEREST_RATE', 'INTEREST_RATE_EXPENSE', 'MARG
IN', 'NET_INTEREST_INCOME']
```

In [100…
```
print("\nCLIENT PORTFOLIO data types:")
print(client_portfolio.dtypes)
```

```
CLIENT PORTFOLIO data types:
CLIENT_NUMBER                      int64
ACCOUNT_OPEN_DATE          datetime64[ns]
OFFICE_PROV                        object
OFFICE_COUNTRY                     object
INDUSTRY                           object
KYC_RATING                         object
CREDIT_RISK_RATING                 object
DEPOSIT_CURRENT_BALANCE           float64
DEP_AVE_YEAR_BAL                  float64
DEP_INT_INCOME                    float64
DEP_INT_EXP                       float64
DEP_NET_INT_INC                   float64
LOAN_CUR_BAL                      float64
LOAN_INT_INCOME                   float64
LOAN_EXPENSE_INTEREST             float64
LOAN_NET_INT_INC                  float64
FEE_INC                           float64
dtype: object
```

In [101…
```
print("\nDEPOSIT ACCOUNT data types:")
print(deposit_account.dtypes)
```

```
DEPOSIT ACCOUNT data types:
CLIENT_NUMBER                      int64
DEPOSIT_ACCOUNT_NO                 int64
DEMAND_TERM                       object
BAL_XCA                          float64
TERM_START_DATE          datetime64[ns]
TERM_END_DATE            datetime64[ns]
TERM_DAYS                         float64
CLIENT_INTEREST_RATE              float64
INTEREST_RATE_EXPENSE             float64
MARGIN                            float64
NET_INTEREST_INCOME               float64
dtype: object
```

Validate and correct data types

In [102…
```
# Convert customer and account identifiers to strings
# This avoids treating identifiers as quantities.
client_portfolio["CLIENT_NUMBER"] = (
    client_portfolio["CLIENT_NUMBER"]
    .astype("Int64")
    .astype("string")
)

deposit_account["CLIENT_NUMBER"] = (
    deposit_account["CLIENT_NUMBER"]
    .astype("Int64")
    .astype("string")
)

deposit_account["DEPOSIT_ACCOUNT_NO"] = (
    deposit_account["DEPOSIT_ACCOUNT_NO"]
    .astype("Int64")
    .astype("string")
)
```

In [103…
```
# Convert date variables
client_portfolio["ACCOUNT_OPEN_DATE"] = pd.to_datetime(
    client_portfolio["ACCOUNT_OPEN_DATE"],
    errors="coerce"
)

deposit_account["TERM_START_DATE"] = pd.to_datetime(
    deposit_account["TERM_START_DATE"],
    errors="coerce"
)

deposit_account["TERM_END_DATE"] = pd.to_datetime(
    deposit_account["TERM_END_DATE"],
    errors="coerce"
)
```

Check duplicate records

In [104…
```
# Count duplicate rows
print(
    "Duplicate CLIENT PORTFOLIO rows:",
    client_portfolio.duplicated().sum()
)
```

```
Duplicate CLIENT PORTFOLIO rows: 0
```

In [105…
```
print(
    "Duplicate DEPOSIT ACCOUNT rows:",
    deposit_account.duplicated().sum()
)
```

```
Duplicate DEPOSIT ACCOUNT rows: 0
```

Clean categorical variables

In [106…
```
# List of categorical columns in CLIENT PORTFOLIO
client_categorical_columns = [
    "OFFICE_PROV",
    "OFFICE_COUNTRY",
    "INDUSTRY",
    "KYC_RATING",
    "CREDIT_RISK_RATING"
]

# Remove leading/trailing spaces and standardize case
for column in client_categorical_columns:
    client_portfolio[column] = (
        client_portfolio[column]
        .astype("string")
        .str.strip()
    )

# Clean deposit type
deposit_account["DEMAND_TERM"] = (
    deposit_account["DEMAND_TERM"]
    .astype("string")
    .str.strip()
    .str.upper()
)
```

Replace blank strings with missing values

In [107…
```
client_portfolio = client_portfolio.replace(
    r"^\s*$",
    np.nan,
    regex=True
)

deposit_account = deposit_account.replace(
    r"^\s*$",
    np.nan,
    regex=True
)
```

In [108…
```
# Give deposit types descriptive names
deposit_account["DEPOSIT_TYPE"] = (
    deposit_account["DEMAND_TERM"]
    .map({
        "D": "Demand Deposit",
        "T": "Term Deposit"
    })
)
print(deposit_account["DEPOSIT_TYPE"].value_counts(dropna=False))
```

```
DEPOSIT_TYPE
Demand Deposit    985
Term Deposit      157
Name: count, dtype: int64
```

Review missing values

In [109…

```
print("Missing values in CLIENT PORTFOLIO:")
display(
    client_portfolio.isna()
    .sum()
    .sort_values(ascending=False)
    .to_frame("Missing_Count")
)

print("Missing values in DEPOSIT ACCOUNT:")
display(
    deposit_account.isna()
    .sum()
    .sort_values(ascending=False)
    .to_frame("Missing_Count")
)
```

```
Missing values in CLIENT PORTFOLIO:
```

	Missing_Count
CREDIT_RISK_RATING	710
KYC_RATING	44
CLIENT_NUMBER	0
DEP_INT_EXP	0
LOAN_NET_INT_INC	0
LOAN_EXPENSE_INTEREST	0
LOAN_INT_INCOME	0
LOAN_CUR_BAL	0
DEP_NET_INT_INC	0
DEP_AVE_YEAR_BAL	0
DEP_INT_INCOME	0
ACCOUNT_OPEN_DATE	0
DEPOSIT_CURRENT_BALANCE	0
INDUSTRY	0
OFFICE_COUNTRY	0
OFFICE_PROV	0
FEE_INC	0

```
Missing values in DEPOSIT ACCOUNT:
```

	Missing_Count
TERM_START_DATE	985
TERM_END_DATE	985
TERM_DAYS	985
CLIENT_NUMBER	0
DEPOSIT_ACCOUNT_NO	0
DEMAND_TERM	0
BAL_XCA	0
CLIENT_INTEREST_RATE	0
INTEREST_RATE_EXPENSE	0
MARGIN	0
NET_INTEREST_INCOME	0
DEPOSIT_TYPE	0

Handle missing values based on business meaning

In [110…
```
# KYC rating
client_portfolio["KYC_RATING"] = (
    client_portfolio["KYC_RATING"]
    .fillna("Not Available")
)
```

In [111…
```
# Credit risk rating: A missing credit risk rating should not automatically be impu
# For non-borrowing clients, it may mean that a credit risk rating is not applicabl
client_portfolio["CREDIT_RISK_RATING"] = np.where(
    client_portfolio["CREDIT_RISK_RATING"].isna()
    & (client_portfolio["LOAN_CUR_BAL"] == 0),
    "Not Applicable",
    client_portfolio["CREDIT_RISK_RATING"]
)

# Remaining missing ratings for clients with loans
client_portfolio["CREDIT_RISK_RATING"] = (
    client_portfolio["CREDIT_RISK_RATING"]
    .fillna("Not Available")
)
```

In [112…
```
# Term deposit fields
# Do not impute dates or terms for demand deposits. Because these fields are struct
# Confirm that missing term information mainly belongs to demand deposits
term_missing_review = deposit_account.groupby("DEPOSIT_TYPE").agg(
    Account_Count=("DEPOSIT_ACCOUNT_NO", "count"),
    Missing_Start_Date=("TERM_START_DATE", lambda x: x.isna().sum()),
    Missing_End_Date=("TERM_END_DATE", lambda x: x.isna().sum()),
```

```
    Missing_Term_Days=("TERM_DAYS", lambda x: x.isna().sum())
)

display(term_missing_review)
```

DEPOSIT_TYPE	Account_Count	Missing_Start_Date	Missing_End_Date	Missing_Term_Days
Demand Deposit	985	985	985	985
Term Deposit	157	0	0	0

Check consistency of categorical values

In [113…

```
for column in [
    "OFFICE_PROV",
    "OFFICE_COUNTRY",
    "INDUSTRY",
    "KYC_RATING",
    "CREDIT_RISK_RATING"
]:
    print(f"\nUnique values in {column}:")
    print(
        client_portfolio[column]
        .value_counts(dropna=False)
    )
```

```
Unique values in OFFICE_PROV:
OFFICE_PROV
ON                  457
BC                  360
AB                  111
QC                   47
DE                    8
Bermuda               7
NB                    7
HONG KONG             6
MB                    5
SK                    4
NS                    4
SINGAPORE             3
TX                    3
NY                    2
Guangdong             2
Zhejiang              2
IL                    1
MA                    1
Beijing               1
FL                    1
London                1
GRAND CAYMAN          1
CIUDAD DE MEXICO      1
Name: count, dtype: Int64

Unique values in OFFICE_COUNTRY:
OFFICE_COUNTRY
CA                995
USA                16
Bermuda             7
HONG KONG           6
China               5
Singapore           3
Cayman Islands      1
UK                  1
Mexico              1
Name: count, dtype: Int64

Unique values in INDUSTRY:
INDUSTRY
Wholesale and Retail Trade                                          312
Real Estate                                                         162
Mining and Oil and Gas Extraction                                   117
Manufacturing                                                       108
Finance and Insurance                                                64
Residential Services, Repairs, and Other Services                    59
Transportation and Warehousing                                       33
Education                                                            28
Information Transmission, Software, and Information Technology Services    26
Agriculture, Forestry, Fishing and Hunting                           23
Public Administration, Social Security, and Social Organizations     22
Culture, Sports, and Entertainment                                   20
Utilities                                                            19
Accommodation and Food Services                                      19
```

```
Scientific Research and Technical Services                          17
Health and Social Work                                               6
Name: count, dtype: Int64

Unique values in KYC_RATING:
KYC_RATING
A                471
C                187
D                182
B                118
Not Available     44
E                 33
Name: count, dtype: Int64

Unique values in CREDIT_RISK_RATING:
CREDIT_RISK_RATING
Not Applicable    706
BB                 65
BB-                62
BB+                48
BBB-               40
B                  26
BBB                17
BBB+               14
B+                 13
B-                 11
CC                  8
CCC                 7
AA                  6
A                   4
Not Available       4
D                   3
AAA                 1
Name: count, dtype: int64
```

In [114…

```
print("Deposit type values:")
print(
    deposit_account[
        ["DEMAND_TERM", "DEPOSIT_TYPE"]
    ].value_counts(dropna=False)
)
```

```
Deposit type values:
DEMAND_TERM  DEPOSIT_TYPE
D            Demand Deposit    985
T            Term Deposit      157
Name: count, dtype: int64
```

Validate numerical fields

In [115…

```
client_numeric_columns = [
    "DEPOSIT_CURRENT_BALANCE",
    "DEP_AVE_YEAR_BAL",
    "DEP_INT_INCOME",
    "DEP_INT_EXP",
    "DEP_NET_INT_INC",
```

```
    "LOAN_CUR_BAL",
    "LOAN_INT_INCOME",
    "LOAN_EXPENSE_INTEREST",
    "LOAN_NET_INT_INC",
    "FEE_INC"
]

deposit_numeric_columns = [
    "BAL_XCA",
    "TERM_DAYS",
    "CLIENT_INTEREST_RATE",
    "INTEREST_RATE_EXPENSE",
    "MARGIN",
    "NET_INTEREST_INCOME"
]

# Convert financial variables to numeric
for column in client_numeric_columns:
    client_portfolio[column] = pd.to_numeric(
        client_portfolio[column],
        errors="coerce"
    )

for column in deposit_numeric_columns:
    deposit_account[column] = pd.to_numeric(
        deposit_account[column],
        errors="coerce"
    )
```

In [116…

```
# Review negative financial values
negative_value_summary = pd.DataFrame({
    "Variable": client_numeric_columns,
    "Negative_Count": [
        (client_portfolio[column] < 0).sum()
        for column in client_numeric_columns
    ]
})

display(negative_value_summary)
```

	Variable	Negative_Count
0	DEPOSIT_CURRENT_BALANCE	0
1	DEP_AVE_YEAR_BAL	0
2	DEP_INT_INCOME	0
3	DEP_INT_EXP	4
4	DEP_NET_INT_INC	16
5	LOAN_CUR_BAL	0
6	LOAN_INT_INCOME	0
7	LOAN_EXPENSE_INTEREST	0
8	LOAN_NET_INT_INC	18
9	FEE_INC	7

In [117…
```
deposit_negative_summary = pd.DataFrame({
    "Variable": deposit_numeric_columns,
    "Negative_Count": [
        (deposit_account[column] < 0).sum()
        for column in deposit_numeric_columns
    ]
})

display(deposit_negative_summary)
```

	Variable	Negative_Count
0	BAL_XCA	0
1	TERM_DAYS	0
2	CLIENT_INTEREST_RATE	0
3	INTEREST_RATE_EXPENSE	0
4	MARGIN	20
5	NET_INTEREST_INCOME	26

These records are retained because negative profitability is a meaningful finding rather than a data error.

Validate calculated fields

In [118…
```
# Check whether the provided net interest income approximately agrees with income m
client_portfolio["CALCULATED_DEP_NII"] = (
    client_portfolio["DEP_INT_INCOME"]
    - client_portfolio["DEP_INT_EXP"]
)
```

```
client_portfolio["CALCULATED_LOAN_NII"] = (
    client_portfolio["LOAN_INT_INCOME"]
    - client_portfolio["LOAN_EXPENSE_INTEREST"]
)

client_portfolio["DEP_NII_DIFFERENCE"] = (
    client_portfolio["DEP_NET_INT_INC"]
    - client_portfolio["CALCULATED_DEP_NII"]
)

client_portfolio["LOAN_NII_DIFFERENCE"] = (
    client_portfolio["LOAN_NET_INT_INC"]
    - client_portfolio["CALCULATED_LOAN_NII"]
)
display(
    client_portfolio[
        [
            "CLIENT_NUMBER",
            "DEP_NET_INT_INC",
            "CALCULATED_DEP_NII",
            "DEP_NII_DIFFERENCE",
            "LOAN_NET_INT_INC",
            "CALCULATED_LOAN_NII",
            "LOAN_NII_DIFFERENCE"
        ]
    ].head()
)
```

	CLIENT_NUMBER	DEP_NET_INT_INC	CALCULATED_DEP_NII	DEP_NII_DIFFERENCE	LOAN_NI
0	10008077214	28863.791195	28863.801195	-0.010000	
1	10008077178	5684.915486	5684.895486	0.020000	
2	10008077209	24060.311968	24060.281969	0.029999	
3	10008076426	18885.643278	18885.643278	0.000000	
4	10008077714	56.777200	56.777200	0.000000	

Small differences is due to rounding or internal allocation methodologies and should be documented rather than automatically overwritten.

Check for potential outliers

In [119…

```
financial_summary = client_portfolio[
    client_numeric_columns
].describe().T

financial_summary["IQR"] = (
    financial_summary["75%"]
    - financial_summary["25%"]
)
```

```
display(financial_summary)
```

	count	mean	std	min	25%	
DEPOSIT_CURRENT_BALANCE	1035.0	1.489296e+06	1.841488e+07	0.000000e+00	0.000	86
DEP_AVE_YEAR_BAL	1035.0	1.693820e+06	1.924637e+07	0.000000e+00	0.000	254
DEP_INT_INCOME	1035.0	6.800060e+04	8.648146e+05	0.000000e+00	0.000	8
DEP_INT_EXP	1035.0	6.294155e+04	8.578802e+05	-1.785620e+03	0.000	
DEP_NET_INT_INC	1035.0	5.059056e+03	4.705680e+04	-6.042473e+05	0.000	6
LOAN_CUR_BAL	1035.0	2.546994e+06	1.745619e+07	0.000000e+00	0.000	
LOAN_INT_INCOME	1035.0	1.614960e+05	9.222988e+05	0.000000e+00	0.000	
LOAN_EXPENSE_INTEREST	1035.0	1.177643e+05	8.377227e+05	0.000000e+00	0.000	
LOAN_NET_INT_INC	1035.0	4.373171e+04	2.868861e+05	-2.004808e+06	0.000	
FEE_INC	1035.0	3.085513e+04	1.803160e+05	-5.664398e+05	54.805	23

In [120…

```
# Create an IQR-based outlier flag for major profitability variables
def create_iqr_flag(data, column):
    q1 = data[column].quantile(0.25)
    q3 = data[column].quantile(0.75)
    iqr = q3 - q1

    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr

    return (
        (data[column] < lower_bound)
        | (data[column] > upper_bound)
    ).astype(int)


for column in [
    "DEP_NET_INT_INC",
    "LOAN_NET_INT_INC",
    "FEE_INC"
]:
    client_portfolio[f"{column}_OUTLIER_FLAG"] = (
        create_iqr_flag(client_portfolio, column)
    )
```

We do not automatically remove these records because major corporate clients may legitimately generate unusually large balances or revenue.

Aggregate deposit accounts to the client level

In [121…
```
deposit_client_summary = (
    deposit_account
    .groupby("CLIENT_NUMBER", as_index=False)
    .agg(
        DEPOSIT_ACCOUNT_COUNT=(
            "DEPOSIT_ACCOUNT_NO",
            "nunique"
        ),
        TOTAL_ACCOUNT_BALANCE=(
            "BAL_XCA",
            "sum"
        ),
        TOTAL_ACCOUNT_NII=(
            "NET_INTEREST_INCOME",
            "sum"
        ),
        AVERAGE_CLIENT_RATE=(
            "CLIENT_INTEREST_RATE",
            "mean"
        ),
        AVERAGE_MARGIN=(
            "MARGIN",
            "mean"
        ),
        DEMAND_ACCOUNT_COUNT=(
            "DEPOSIT_TYPE",
            lambda x: (x == "Demand Deposit").sum()
        ),
        TERM_ACCOUNT_COUNT=(
            "DEPOSIT_TYPE",
            lambda x: (x == "Term Deposit").sum()
        )
    )
)

display(deposit_client_summary.head())
```

	CLIENT_NUMBER	DEPOSIT_ACCOUNT_COUNT	TOTAL_ACCOUNT_BALANCE	TOTAL_ACCOUN
0	10002000011	2	1.515688e+03	69.02
1	10002000031	4	5.375191e+06	34849.39
2	10002000058	2	2.904088e+05	10896.50
3	10002000489	1	5.490800e+08	11518.95
4	10002000553	3	1.438012e+07	278416.72

Perform the full outer join

In [122…
```
integrated_data = pd.merge(
    client_portfolio,
    deposit_client_summary,
```

```
    on="CLIENT_NUMBER",
    how="outer",
    indicator=True
)

print(integrated_data["_merge"].value_counts())
```

```
_merge
both          643
left_only     392
right_only      1
Name: count, dtype: int64
```

Create relationship indicators

In [123…

```
# Replace missing balances with zero for relationship classification
integrated_data[
    [
        "DEPOSIT_CURRENT_BALANCE",
        "TOTAL_ACCOUNT_BALANCE",
        "LOAN_CUR_BAL"
    ]
] = integrated_data[
    [
        "DEPOSIT_CURRENT_BALANCE",
        "TOTAL_ACCOUNT_BALANCE",
        "LOAN_CUR_BAL"
    ]
].fillna(0)
```

In [124…

```
# Determine whether the client has a deposit relationship
integrated_data["HAS_DEPOSIT"] = (
    (integrated_data["DEPOSIT_CURRENT_BALANCE"] > 0)
    | (integrated_data["TOTAL_ACCOUNT_BALANCE"] > 0)
    | (integrated_data["DEPOSIT_ACCOUNT_COUNT"].fillna(0) > 0)
).astype(int)

# Determine whether the client has a loan relationship
integrated_data["HAS_LOAN"] = (
    integrated_data["LOAN_CUR_BAL"] > 0
).astype(int)
```

In [125…

```
# Create relationship categories
conditions = [
    (integrated_data["HAS_DEPOSIT"] == 1)
    & (integrated_data["HAS_LOAN"] == 1),

    (integrated_data["HAS_DEPOSIT"] == 1)
    & (integrated_data["HAS_LOAN"] == 0),

    (integrated_data["HAS_DEPOSIT"] == 0)
    & (integrated_data["HAS_LOAN"] == 1)
]

relationship_categories = [
```

```
    "Full Relationship",
    "Deposit Only",
    "Loan Only"
]

integrated_data["RELATIONSHIP_TYPE"] = np.select(
    conditions,
    relationship_categories,
    default="No Active Balance"
)

print(
    integrated_data["RELATIONSHIP_TYPE"]
    .value_counts(dropna=False)
)
```

```
RELATIONSHIP_TYPE
Deposit Only         627
No Active Balance    334
Loan Only             58
Full Relationship     17
Name: count, dtype: int64
```

In [126…

```
# Create binary variables
integrated_data["DEPOSIT_ONLY_FLAG"] = (
    integrated_data["RELATIONSHIP_TYPE"]
    == "Deposit Only"
).astype(int)

integrated_data["LOAN_ONLY_FLAG"] = (
    integrated_data["RELATIONSHIP_TYPE"]
    == "Loan Only"
).astype(int)

integrated_data["FULL_RELATIONSHIP_FLAG"] = (
    integrated_data["RELATIONSHIP_TYPE"]
    == "Full Relationship"
).astype(int)
```

In [127…

```
# Final checks
print("Final integrated dataset shape:")
print(integrated_data.shape)

print("\nDuplicate client IDs:")
print(integrated_data["CLIENT_NUMBER"].duplicated().sum())

print("\nMissing customer IDs:")
print(integrated_data["CLIENT_NUMBER"].isna().sum())

print("\nRelationship type distribution:")
print(integrated_data["RELATIONSHIP_TYPE"].value_counts())

print("\nMerge source distribution:")
print(integrated_data["_merge"].value_counts())
```

```
Final integrated dataset shape:
(1036, 38)

Duplicate client IDs:
0

Missing customer IDs:
0

Relationship type distribution:
RELATIONSHIP_TYPE
Deposit Only         627
No Active Balance    334
Loan Only             58
Full Relationship     17
Name: count, dtype: int64

Merge source distribution:
_merge
both          643
left_only     392
right_only      1
Name: count, dtype: int64
```

In [128…

```
display(
    integrated_data[
        [
            "CLIENT_NUMBER",
            "INDUSTRY",
            "KYC_RATING",
            "CREDIT_RISK_RATING",
            "DEPOSIT_CURRENT_BALANCE",
            "LOAN_CUR_BAL",
            "DEPOSIT_ACCOUNT_COUNT",
            "TOTAL_ACCOUNT_BALANCE",
            "RELATIONSHIP_TYPE"
        ]
    ].head(20)
)
```

	CLIENT_NUMBER	INDUSTRY	KYC_RATING	CREDIT_RISK_RATING	DEPOSIT_CURRENT_
0	10002000002	Finance and Insurance	Not Available	Not Applicable	0.00
1	10002000009	Real Estate	A	BBB-	0.00
2	10002000011	Transportation and Warehousing	C	Not Applicable	1.51
3	10002000014	Real Estate	A	BB-	0.00
4	10002000016	Real Estate	C	BB-	0.00
5	10002000018	Real Estate	C	BB-	0.00
6	10002000028	Finance and Insurance	C	BBB-	0.00
7	10002000029	Real Estate	A	BB+	0.00
8	10002000031	Manufacturing	E	Not Applicable	5.37
9	10002000034	Finance and Insurance	Not Available	BB-	0.00
10	10002000038	Manufacturing	A	BB-	0.00
11	10002000039	Mining and Oil and Gas Extraction	A	BBB-	0.00
12	10002000041	Culture, Sports, and Entertainment	A	BB-	0.00
13	10002000042	Manufacturing	A	BB-	0.00
14	10002000043	Manufacturing	A	BB	0.00
15	10002000044	Finance and Insurance	B	A	0.00
16	10002000049	Manufacturing	B	BBB	0.00
17	10002000052	Finance and Insurance	B	BBB-	0.00
18	10002000056	Residential Services, Repairs, and Other Services	Not Available	Not Applicable	0.00
19	10002000057	Manufacturing	A	Not Applicable	0.00

Export the cleaned datasets

In [129…

```
output_path = r"C:\Users\BOC\Desktop\MFIT 860\Project\Data\cleaned_project_data.xls

with pd.ExcelWriter(
    output_path,
    engine="openpyxl"
) as writer:

    client_portfolio.to_excel(
        writer,
        sheet_name="CLEAN_CLIENT_PORTFOLIO",
        index=False
    )

    deposit_account.to_excel(
        writer,
        sheet_name="CLEAN_DEPOSIT_ACCOUNT",
        index=False
    )

    deposit_client_summary.to_excel(
        writer,
        sheet_name="DEPOSIT_CLIENT_SUMMARY",
        index=False
    )

    integrated_data.to_excel(
        writer,
        sheet_name="INTEGRATED_DATA",
        index=False
    )

print("Cleaned file saved to:")
print(output_path)
```

```
Cleaned file saved to:
C:\Users\BOC\Desktop\MFIT 860\Project\Data\cleaned_project_data.xlsx
```

In [ ]: